

From Exhaustive Search to Time–Memory Tradeoffs: Reproducible Hellman and Distinguished-Point Experiments over Reduced DES Keyspaces

Ali Mkhida

Updated research note based on an original M2P SCCI project
with Abdourahmane Sakho and Maad El Yadari

August 2026

Abstract

This note revisits a historical cryptography coursework project and isolates one of its algorithmic cores: exhaustive key search and time–memory tradeoffs. Instead of treating four raw DES bytes as a nominal 32-bit search space, the updated implementation works with explicit reduced state spaces built from the effective DES key bits after parity is removed. We define a deterministic functional graph

$$F(x) = R(E_{K(x)}(P)),$$

and study exhaustive search, fixed-length Hellman chains, and bounded distinguished-point chains as different ways of exploring this graph. The modern Java reference implementation adds deterministic tests, indexed endpoint lookup, exact candidate verification, seeded experiments, and exact unique-state coverage analysis. In a reproducible 16-bit experiment with 30 seeded trials, Hellman tables with $m = 1024$ and $t = 64$ cover a median of 16.8% of the state space, while distinguished-point tables cover 15.4%. These measurements also show why the independent-sample occupancy quantity $1 - e^{-mt/N}$ should not be interpreted as the expected coverage of iterated chains: coalescence in a fixed functional graph is substantial. The Java implementation is intended as a reference specification for a later Rust implementation and cross-language performance study.

1 Introduction

Exhaustive key search is the simplest baseline for recovering a key from a known plaintext–ciphertext pair: test candidate keys until one reproduces the observed ciphertext. If the candidate space has size N , the online work is linear in N in the worst case and approximately $N/2$ trials for a uniformly sampled target.

Time–memory tradeoff methods move part of that work into an offline phase. Hellman’s classical construction stores only selected chain endpoints rather than a full table of all key–ciphertext pairs [1]. A related engineering idea is to terminate variable-length chains at distinguished points, which provides a compact representation and is particularly useful in parallel collision and cryptanalytic searches [2].

The original project associated with this repository mixed several topics: finite-field basis transformations, hardware-oriented AES material, and a practical Java implementation of DES key-search experiments. The present note does not attempt to reconstruct all of those themes. Instead, it turns the practical-work component into a small reproducible study with an explicit state model, a functional-graph formulation, verified algorithms, and measured table coverage.

2 From a Raw DES Representation to an Explicit State Space

DES keys are represented by eight bytes, but one bit in each byte is a parity bit. Consequently, four varying raw bytes do not constitute four independent bytes of effective DES key material. The historical prototype varied four raw bytes, which makes statements such as “32 bits of entropy” ambiguous when interpreted as effective DES key bits.

The updated implementation instead chooses an integer state dimension b and defines

$$\mathcal{X}_b = \{0, 1, \dots, 2^b - 1\}, \quad 1 \leq b \leq 28.$$

Each state $x \in \mathcal{X}_b$ is encoded into four groups of seven effective DES bits; odd parity is then inserted to obtain a canonical DES key encoding $K(x)$. The first four DES bytes are fixed, while the final four bytes carry at most $4 \cdot 7 = 28$ effective variable bits.

This distinction between *raw representation bits* and *effective state bits* is important for both correctness and reproducibility. All modern experiments therefore report b and $N = 2^b$ directly.

3 Functional-Graph Formulation

Fix a plaintext P . Let $E_{K(x)}(P)$ denote DES encryption of P under the key encoded from state x . The reduction function R maps a ciphertext back into the reduced state space by reading the final four ciphertext bytes, removing the DES parity-bit position from each byte, assembling the resulting 28-bit value, and masking to b bits. We then obtain the deterministic map

$$F : \mathcal{X}_b \rightarrow \mathcal{X}_b, \quad F(x) = R(E_{K(x)}(P)). \quad (1)$$

Because every state has one outgoing edge, F defines a directed functional graph: trees feed into cycles. Hellman and distinguished-point tables can be viewed as compact descriptions of trajectories in this graph. This viewpoint also makes chain merging unavoidable rather than exceptional: once two chains reach the same state, all subsequent states are identical.

4 Search Strategies

4.1 Exhaustive search

Given a target ciphertext C^* , exhaustive search tests states in the configured domain until

$$E_{K(x)}(P) = C^*.$$

It uses negligible precomputed memory, has no offline phase, and provides a useful correctness and timing baseline. In the curated experiment the complete reduced state space is therefore represented, so its exact coverage is 100%.

4.2 Hellman chains

Choose m distinct start states $s_1, \dots, s_m$. For a fixed chain length t , compute

$$s_i, F(s_i), F^2(s_i), \dots, F^t(s_i)$$

and store the pair consisting of the start state and final endpoint. During online lookup, the target is projected toward possible endpoints; matching stored endpoints identify candidate chains, which are regenerated and verified against the original target ciphertext.

The historical Java prototype repeatedly scanned the endpoint list during online lookup. The modern implementation instead indexes endpoints in a hash map and retains all starts that collide at the same endpoint. This changes the engineering complexity of lookup without changing the underlying chain construction.

If mt states were independent uniform samples from a set of size N , their occupancy would be approximated by

$$1 - e^{-mt/N}. \quad (2)$$

This quantity is useful only as an independent-sampling reference here. The states in a Hellman chain are not independent: they are consecutive iterates of one fixed map F . Chain coalescence can therefore reduce coverage far below (2).

4.3 Distinguished-point chains

Let d be a distinguished-point difficulty. In the modern implementation a state is distinguished when its lowest d state bits are zero. Under a uniform-state heuristic, the waiting time to such a state is approximately 2^d transitions.

A chain begins at a sampled start state and is iterated until either a distinguished state is reached or a configured maximum chain length is exceeded. Only chains that reach a distinguished endpoint are stored. Multiple chains may share the same endpoint, so online lookup again regenerates all candidate chains and verifies the target ciphertext before returning a state.

The historical implementation contained a branch that verified one candidate key but returned a different state variable. The updated implementation removes that ambiguity and includes regression tests that require the returned state to be the state actually verified against the target ciphertext.

5 Reproducible Implementation

The modern Java 17 implementation separates the state model, DES oracle, search algorithms, and experiment harness. The main changes relative to the historical prototype are:

- an explicit effective-bit state model with canonical DES parity;
- deterministic, seedable table generation;
- endpoint hash indexing for Hellman and distinguished-point tables;
- retention of endpoint collisions rather than silently overwriting them;
- exact candidate-chain regeneration and ciphertext verification;
- bounded distinguished-point chains with explicit truncation counts;
- automated tests for state encoding, recovery, negative cases, determinism, truncation, and coverage;
- raw CSV experiment output and recorded runtime environment metadata;
- exact unique-state coverage computed outside the timed benchmark path.

The last item is especially important. Random-target recovery over a small number of trials is noisy, whereas exact coverage directly measures how much of the reduced state space is represented by a generated table.

Coverage and success probability. Let $S \subseteq \mathcal{X}_b$ be the union of states represented by a table and assume the lookup procedure recovers a target exactly when the target state lies in S . If the target X is uniform on $\mathcal{X}_b$, then

$$\Pr[X \text{ is recovered}] = \frac{|S|}{2^b}.$$

Thus exact coverage is the natural population quantity; finite random-target recovery is an empirical check of the implementation and sampling process.

6 Verified Experiment

The curated experiment uses a 16-bit effective state space,

$$N = 2^{16} = 65,536,$$

and 30 deterministic seeded trials. A separate warm-up run precedes the timed measurements. Timings are end-to-end JVM observations obtained with `System.nanoTime()`; they are not cycle counts or JMH microbenchmarks.

For Hellman tables we use $m = 1024$ start states and $t = 64$ transitions, so $mt/N = 1$. Distinguished-point tables use 1024 starts, difficulty $d = 6$, and a maximum chain length of 256 transitions.

Table 1: Curated 16-bit reduced-state experiment. Coverage is computed exactly from the generated table chains. Timing values are medians over 30 seeded trials and are machine-dependent.

Method	Exact recovery	Median coverage	Offline	Online
Exhaustive	30/30	100.0%	0.000 ms	25.863 ms
Hellman	4/30	16.8%	70.222 ms	3.399 ms
Distinguished points	3/30	15.4%	59.311 ms	1.790 ms

For Hellman tables, exact coverage ranges from 16.4% to 17.4%, with a median of 690.5 distinct endpoints out of 1024 stored chains. The independent-sample quantity in (2) equals

$$1 - e^{-1} \approx 63.2\%,$$

but the measured exact coverage is only 16.8%. The gap is not evidence of a failed benchmark: it exposes how strongly trajectories coalesce when the same functional graph is iterated repeatedly.

Distinguished-point tables show a similar effect. Their exact coverage ranges from 15.1% to 15.8%, while the median number of distinct distinguished endpoints is only 144.5 out of 1024 starts. Across all 30 trials, 30,619 chains are stored and 101 chains are truncated by the maximum-length bound.

7 Discussion

The experiment illustrates a distinction that can be hidden when a time-memory tradeoff is summarized only through m , t , and N . The nominal product mt does not itself measure represented state-space coverage. The geometry of the functional graph matters: trajectories can merge early, after which additional steps add no new states relative to another chain.

This suggests several natural follow-up experiments. Different reduction functions can alter the induced functional graph; multiple tables with distinct reductions can reduce systematic merging; and parameter sweeps can compare memory, preprocessing time, online time, endpoint diversity, and exact coverage rather than relying on one nominal tradeoff equation.

The results should not be interpreted as a security estimate for DES. DES is obsolete, and the state space here is deliberately small. The value of the experiment is methodological: the reduced domain permits complete coverage analysis and exact reproducibility while preserving the structure of the search algorithms.

8 Toward a Rust Implementation

The current Java code should be treated as a reference semantics rather than the final performance implementation. The next implementation phase is a Rust port with cross-language equivalence tests. For a fixed plaintext, state dimension, seed, reduction, and parameter set, Java and Rust should produce the same canonical key encodings, chain transitions, endpoints, exact coverage, and recovery decisions.

Only after those equivalence tests pass should performance comparisons be made. This separates two questions that are otherwise easy to conflate:

1. does the implementation realize the intended time–memory tradeoff algorithm correctly?;
2. how efficiently does a particular language/runtime realize it?

The Rust phase can then explore allocation control, compact table layouts, parallel chain construction, and SIMD-friendly batching without losing the reference behavior established here.

9 Conclusion

This update turns a historical DES practical exercise into a compact reproducible study of cryptanalytic time–memory tradeoffs. The central objects are no longer opaque Java byte arrays but an explicit reduced state space and the deterministic functional graph induced by encryption and reduction. Exact coverage measurements show substantial chain coalescence: at $mt/N = 1$, the curated Hellman tables cover a median of only 16.8% of the state space, while distinguished-point tables cover 15.4%. Preserving the original presentation and prototype separately from this note makes the historical provenance clear, while the modern implementation provides a tested reference point for the planned Rust translation.

References

- [1] M. E. Hellman, “A cryptanalytic time–memory trade-off,” *IEEE Transactions on Information Theory*, vol. 26, no. 4, pp. 401–406, 1980. DOI: 10.1109/TIT.1980.1056220.
- [2] P. C. van Oorschot and M. J. Wiener, “Parallel collision search with cryptanalytic applications,” *Journal of Cryptology*, vol. 12, no. 1, pp. 1–28, 1999.